

```
// =====
// 文件: main.dart
// 原始行数: 156 | 保留行数: 69
// =====

import 'dart:async';
import 'dart:io';
import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:flutter_localizations/flutter_localizations.dart';
import 'package:provider/provider.dart';
import 'package:shared_preferences/shared_preferences.dart';
import 'package:sqflite_common_ffi/sqflite_ffi.dart';
import 'package:http/http.dart' as http;
import 'ai/tencentcloud/embedded_public_hunyuan_credentials.dart';
import 'ai/tencentcloud/tencent_credentials.dart';
import 'ai/config/auth_service.dart';
import 'ai/config/checkin_service.dart';
import 'ai/config/remote_config_service.dart';
import 'ai/tencentcloud/tencent_api_client.dart';
import 'core/theme/app_theme.dart';
import 'presentation/pages/bookshelf/bookshelf_page.dart';
import 'presentation/providers/ai_model_provider.dart';
import 'presentation/providers/books_provider.dart';
import 'presentation/providers/read_aloud_provider.dart';
import 'presentation/providers/translation_provider.dart';
import 'presentation/providers/qa_stream_provider.dart';
import 'presentation/providers/illustration_provider.dart';

import 'package:flutter/services.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  // Set global system UI mode to edge-to-edge for modern Android experience
  SystemChrome.setEnabledSystemUIMode(SystemUiMode.edgeToEdge);
  SystemChrome.setSystemUIOverlayStyle(const SystemUiOverlayStyle(

// =====
// ... main.dart 中间部分代码省略 (92 行) ...
// =====

  return MaterialApp(
    title: '灵阅',
    debugShowCheckedModeBanner: false,
    theme: AppTheme.lightTheme,
    darkTheme: AppTheme.darkTheme,
    themeMode: ThemeMode.system,
    localizationsDelegates: const [
      GlobalMaterialLocalizations.delegate,
      GlobalWidgetsLocalizations.delegate,
      GlobalCupertinoLocalizations.delegate,
    ],
    supportedLocales: const [
```

```

        Locale('zh', 'CN'),
        Locale('en', 'US'),
    ],
    home: const BookshelfPage(),
  );
}
}

/// 请求网络权限 (iOS)
Future<void> _requestNetworkPermission() async {
  try {
    // 发送一个简单的网络请求来触发权限弹窗
    await http
      .get(Uri.parse('https://www.baidu.com'))
      .timeout(const Duration(seconds: 5));
  } catch (e) {
    // 忽略错误, 只需要触发权限请求
  }
}

// =====
// 文件: data/models/book.dart
// 原始行数: 120 | 保留行数: 120
// =====

import 'dart:typed_data';

class Book {
  final String id;
  final String title;
  final String author;
  final String coverPath;
  final String filePath;
  final String format;
  final int totalPages;
  final DateTime importDate;
  final int currentPage;
  final double percentage;
  final DateTime? lastRead;
  final int readingChapter;
  final int readingPage;
  final double readingProgress;
  final Uint8List? coverBytes;
  final Uint8List? fileBytes;

  Book({
    required this.id,
    required this.title,
    required this.author,
    this.coverPath = '',
    required this.filePath,
    required this.format,
    this.totalPages = 0,
    required this.importDate,

```

```

    this.currentPage = 0,
    this.percentage = 0.0,
    this.lastRead,
    this.readingChapter = 0,
    this.readingPage = 0,
    this.readingProgress = 0.0,
    this.coverBytes,
    this.fileBytes,
  });

```

```

// Convert to Map for Database

```

```

Map<String, dynamic> toMap() {
  return {
    'id': id,
    'title': title,
    'author': author,
    'cover_path': coverPath,
    'file_path': filePath,
    'format': format,
    'total_pages': totalPages,
    'import_date': importDate.millisecondsSinceEpoch,
    'current_page': currentPage,
    'percentage': percentage,
    'last_read': lastRead?.millisecondsSinceEpoch,
    'reading_chapter': readingChapter,
    'reading_page': readingPage,
    'reading_progress': readingProgress,
  };
}

```

```

// Create from Map

```

```

factory Book.fromMap(Map<String, dynamic> map) {
  return Book(
    id: map['id'],
    title: map['title'],
    author: map['author'] ?? '',
    coverPath: map['cover_path'] ?? '',
    filePath: map['file_path'] ?? '',
    format: map['format'] ?? 'unknown',
    totalPages: map['total_pages'] ?? 0,
    importDate: DateTime.fromMillisecondsSinceEpoch(map['import_date']),
    currentPage: map['current_page'] ?? 0,
    percentage: map['percentage'] ?? 0.0,
    lastRead: map['last_read'] != null
      ? DateTime.fromMillisecondsSinceEpoch(map['last_read'])
      : null,
    readingChapter: map['reading_chapter'] ?? 0,
    readingPage: map['reading_page'] ?? 0,
    readingProgress: (map['reading_progress'] ?? 0.0) * 1.0,
  );
}

```

```

Book copyWith({
  String? id,
  String? title,

```

```

    String? author,
    String? coverPath,
    String? filePath,
    String? format,
    int? totalPages,
    DateTime? importDate,
    int? currentPage,
    double? percentage,
    DateTime? lastRead,
    int? readingChapter,
    int? readingPage,
    double? readingProgress,
    Uint8List? coverBytes,
    Uint8List? fileBytes,
  }) {
    return Book(
      id: id ?? this.id,
      title: title ?? this.title,
      author: author ?? this.author,
      coverPath: coverPath ?? this.coverPath,
      filePath: filePath ?? this.filePath,
      format: format ?? this.format,
      totalPages: totalPages ?? this.totalPages,
      importDate: importDate ?? this.importDate,
      currentPage: currentPage ?? this.currentPage,
      percentage: percentage ?? this.percentage,
      lastRead: lastRead ?? this.lastRead,
      readingChapter: readingChapter ?? this.readingChapter,
      readingPage: readingPage ?? this.readingPage,
      readingProgress: readingProgress ?? this.readingProgress,
      coverBytes: coverBytes ?? this.coverBytes,
      fileBytes: fileBytes ?? this.fileBytes,
    );
  }
}

// =====
// 文件: data/database/database_helper.dart
// 原始行数: 153 | 保留行数: 153
// =====

import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';
import '../models/book.dart';

class DatabaseHelper {
  static final DatabaseHelper instance = DatabaseHelper._init();
  static Database? _database;

  DatabaseHelper._init();

  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await _initDB('airread.db');
  }
}

```

```

    return _database!;
}

Future<Database> _initDB(String filePath) async {
    final dbPath = await getDatabasesPath();
    final path = join(dbPath, filePath);

    return await openDatabase(path,
        version: 4, onCreate: _createDB, onUpgrade: _upgradeDB);
}

Future _createDB(Database db, int version) async {
    const bookTable = '''
CREATE TABLE books (
    id TEXT PRIMARY KEY,
    title TEXT NOT NULL,
    author TEXT,
    cover_path TEXT,
    file_path TEXT NOT NULL,
    format TEXT NOT NULL,
    total_pages INTEGER DEFAULT 0,
    import_date INTEGER NOT NULL,
    current_page INTEGER DEFAULT 0,
    percentage REAL DEFAULT 0.0,
    last_read INTEGER,
    reading_chapter INTEGER DEFAULT 0,
    reading_page INTEGER DEFAULT 0,
    reading_progress REAL DEFAULT 0.0
)
''';

    const settingsTable = '''
CREATE TABLE settings (
    key TEXT PRIMARY KEY,
    value TEXT
)
''';

    await db.execute(bookTable);
    await db.execute(settingsTable);
}

Future _upgradeDB(Database db, int oldVersion, int newVersion) async {
    if (oldVersion < 2) {
        const settingsTable = '''
CREATE TABLE IF NOT EXISTS settings (
    key TEXT PRIMARY KEY,
    value TEXT
)
''';
        await db.execute(settingsTable);
    }
    if (oldVersion < 3) {
        try {
            await db.execute(

```

```

        'ALTER TABLE books ADD COLUMN reading_chapter INTEGER DEFAULT 0');
    } catch (_) {}
    try {
        await db.execute(
            'ALTER TABLE books ADD COLUMN reading_page INTEGER DEFAULT 0');
    } catch (_) {}
}
if (oldVersion < 4) {
    try {
        await db.execute(
            'ALTER TABLE books ADD COLUMN reading_progress REAL DEFAULT 0.0');
    } catch (_) {}
}
}
}

```

```

Future<void> insertBook(Book book) async {
    final db = await instance.database;
    await db.insert(
        'books',
        book.toMap(),
        conflictAlgorithm: ConflictAlgorithm.replace,
    );
}

```

```

Future<List<Book>> getAllBooks() async {
    final db = await instance.database;
    final result = await db.query('books', orderBy: 'import_date DESC');
    return result.map((json) => Book.fromMap(json)).toList();
}

```

```

Future<Book?> getBook(String id) async {
    final db = await instance.database;
    final maps = await db.query(
        'books',
        columns: null,
        where: 'id = ?',
        whereArgs: [id],
    );

    if (maps.isNotEmpty) {
        return Book.fromMap(maps.first);
    } else {
        return null;
    }
}

```

```

Future<int> updateBook(Book book) async {
    final db = await instance.database;
    return db.update(
        'books',
        book.toMap(),
        where: 'id = ?',
        whereArgs: [book.id],
    );
}

```

```

Future<int> deleteBook(String id) async {
  final db = await instance.database;
  return await db.delete(
    'books',
    where: 'id = ?',
    whereArgs: [id],
  );
}

Future<void> saveReadingSettings(double fontSize, double lineHeight) async {
  final db = await instance.database;
  await db.insert(
    'settings',
    {'key': 'fontSize', 'value': fontSize.toString()},
    conflictAlgorithm: ConflictAlgorithm.replace,
  );
  await db.insert(
    'settings',
    {'key': 'lineHeight', 'value': lineHeight.toString()},
    conflictAlgorithm: ConflictAlgorithm.replace,
  );
}

Future<void> close() async {
  final db = await instance.database;
  db.close();
}
}

// =====
// 文件: presentation/providers/books_provider.dart
// 原始行数: 333 | 保留行数: 165
// =====

import 'package:flutter/foundation.dart';
import '.../data/models/book.dart';
import '.../data/database/database_helper.dart';
import '.../data/database/web_database_helper.dart';
import '.../data/services/book_importer.dart';
import 'translation_provider.dart';

import 'package:file_picker/file_picker.dart';

class BooksProvider extends ChangeNotifier {
  List<Book> _books = [];
  bool _isLoading = false;
  bool _isImporting = false;
  String _loadingMessage = '';
  String? _importError;

  // Animation Control
  List<String> _recentlyImportedIds = [];
  int _importBatchId = 0;

```

```

int _booksLoadRequestId = 0;

final DatabaseHelper _dbHelper = DatabaseHelper.instance;
final WebDatabaseHelper _webDbHelper = WebDatabaseHelper.instance;
final BookImporter _importer = BookImporter();

// Selection Mode
bool _isSelectionMode = false;
final Set<String> _selectedBookIds = {};

List<Book> get books => _books;
bool get isLoading => _isLoading;
bool get isImporting => _isImporting;
String get loadingMessage => _loadingMessage;
String? get importError => _importError;

bool get isSelectionMode => _isSelectionMode;
Set<String> get selectedBookIds => _selectedBookIds;

List<String> get recentlyImportedIds => _recentlyImportedIds;
int get importBatchId => _importBatchId;

BooksProvider() {
  loadBooks();
}

void _invalidatePendingLoads() {
  _booksLoadRequestId++;
}

void toggleSelectionMode() {
  _isSelectionMode = !_isSelectionMode;
  if (!_isSelectionMode) {
    _selectedBookIds.clear();
  }
  notifyListeners();
}

void toggleBookSelection(String id) {
  if (_selectedBookIds.contains(id)) {
    _selectedBookIds.remove(id);
  } else {
    _selectedBookIds.add(id);
  }
  notifyListeners();
}

void selectAll() {
  if (_selectedBookIds.length == _books.length) {
    _selectedBookIds.clear();
  } else {
    _selectedBookIds.addAll(_books.map((b) => b.id));
  }
  notifyListeners();
}

```

```

Future<void> pinSelectedBooks() async {
  if (_selectedBookIds.isEmpty) return;

  _isLoading = true;
  notifyListeners();

// =====
// ... presentation/providers/books_provider.dart 中间部分代码省略 (173 行) ...
// =====

  // 2. If files picked, start import process (show overlay)
  if (result != null) {
    _isImporting = true;
    _importError = null;
    notifyListeners();

    try {
      final newBooks = await _importer.processPickedFiles(result);
      if (newBooks.isNotEmpty) {
        _invalidatePendingLoads();
        // Update Animation State
        _recentlyImportedIds = newBooks.map((b) => b.id).toList();
        _importBatchId++;

        _books.insertAll(0, newBooks); // Add to top
      }
    } catch (e) {
      debugPrint('Error importing books: $e');
      _importError = '导入失败: $e';
    } finally {
      _isImporting = false;
      notifyListeners();
    }
  }
}

Future<void> deleteBook(String id) async {
  if (kIsWeb) {
    await _webDbHelper.deleteBook(id);
  } else {
    await _dbHelper.deleteBook(id);
  }
  try {
    await TranslationProvider.removeBookScopedPrefs(id);
  } catch (_) {}
  _books.removeWhere((b) => b.id == id);
  notifyListeners();
}

Future<void> saveReadingSettingsToDb({
  required double fontSize,
  required double lineHeight,
}) async {
  if (kIsWeb) return;

```

```

    await _dbHelper.saveReadingSettings(fontSize, lineHeight);
  }

Future<void> saveReadingProgress({
  required String bookId,
  required int chapterIndex,
  required int pageInChapter,
  required double progress,
  double? overallProgress,
}) async {
  final index = _books.indexWhere((b) => b.id == bookId);
  if (index == -1) return;

  final now = DateTime.now();
  final updated = _books[index].copyWith(
    readingChapter: chapterIndex,
    readingPage: pageInChapter,
    readingProgress: progress,
    percentage: overallProgress ?? _books[index].percentage,
    lastRead: now,
  );

  _books[index] = updated;

  try {
    if (kIsWeb) {
      await _webDbHelper.insertBook(updated);
    } else {
      await _dbHelper.updateBook(updated);
    }
  } catch (_) {}

  notifyListeners();
}

// =====
// 文件: presentation/pages/bookshelf/bookshelf_page.dart
// 原始行数: 591 | 保留行数: 64
// =====

import 'dart:async';

import 'package:flutter/material.dart';
import 'package:flutter/foundation.dart';
import 'package:flutter/services.dart';
import 'package:file_picker/file_picker.dart';
import 'package:provider/provider.dart';
import '../providers/books_provider.dart';
import '../widgets/book_card.dart';
import '../widgets/air_title.dart';
import '../widgets/glass_panel.dart';
import '../core/theme/app_colors.dart';
import '../core/theme/app_tokens.dart';

```

```

import '../reader/reader_page.dart';
import '../../ai/config/app_update_service.dart';
import '../../data/models/book.dart';

class BookshelfPage extends StatefulWidget {
  const BookshelfPage({super.key});
  // ... 函数实现省略 ...
}

class _BookshelfPageState extends State<BookshelfPage>
  final TextEditingController _searchController = TextEditingController();
  String _searchQuery = '';

  static const _intentChannel = MethodChannel('airread/intent');

  void initState() {
    // ... 函数实现省略 ...
  }

  void didChangeAppLifecycleState(AppLifecycleState state) {
    // ... 函数实现省略 ...
  }

  Future<void> _checkInitialEpub() async {
    // ... 方法实现省略 ...
  }

  void dispose() {
    // ... 函数实现省略 ...
  }

  Future<void> _handleImport(BuildContext context) async {
    // ... 方法实现省略 ...
  }

  void _onSearchChanged(String value) {
    // ... 函数实现省略 ...
  }

  List<Book> _filterBooks(List<Book> books) {
    // ... 方法实现省略 ...
  }

  Widget build(BuildContext context) {
    // ... 函数实现省略 ...
  }

  Widget _buildActionButton({
    // ... 函数实现省略 ...
  })

// =====
// 文件: presentation/pages/reader/reader_page.dart
// 原始行数: 7759 | 保留行数: 350

```

```
// =====

import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:flutter/foundation.dart';
import 'package:provider/provider.dart';
import 'package:epubx/epubx.dart';
import 'dart:async';
import 'dart:collection';
import 'dart:convert';
import 'dart:io';
import 'package:path_provider/path_provider.dart';
import 'package:audioplayers/audioplayers.dart';
import 'package:shared_preferences/shared_preferences.dart';
import '../core/theme/app_colors.dart';
import '../core/theme/app_tokens.dart';
import '../widgets/ai_hud.dart';
import '../widgets/glass_panel.dart';
import '../providers/books_provider.dart';
import '../providers/ai_model_provider.dart';
import '../providers/read_aloud_provider.dart';
import '../providers/translation_provider.dart';
import '../data/services/book_parser.dart';
import '../data/models/book.dart';
import 'widgets/reader_paragraph.dart';
import '../ai/translation/translation_types.dart';
import '../ai/tencentcloud/embedded_public_hunyuan_credentials.dart';
import '../ai/tencent_tts/tencent_tts_client.dart';
import 'tts_web_speech.dart'

/// 解码 HTML 实体 (如 &#45556;\&#x3000;) , 修复灵译等翻译 EPUB 中的乱码
String _decodeHtmlEntities(String html) {
    // ... 函数实现省略 ...
}

class _MeasureSize extends StatefulWidget {
    final Widget child;
    final ValueChanged<Size> onChange;

    const _MeasureSize({
        // ... 函数实现省略 ...
    })

class _MeasureSizeState extends State<_MeasureSize> {
    Widget build(BuildContext context) {
        // ... 函数实现省略 ...
    }
}

abstract class _ReaderChapter {

class _EpubReaderChapter implements _ReaderChapter {
    final EpubChapterRef ref;
    final String? _titleOverride;
    final bool hasSubChapters;
    final int? parentIndex;
```

```

class _TextReaderChapter implements _ReaderChapter {
    final String _title;
    /// Only holds the chapter's own text (substring), not the entire book.
    final String _chapterText;

    start.clamp(0, sourceText.length),
    end.clamp(start.clamp(0, sourceText.length), sourceText.length),

    Future<String> readPlainText() async {
    // ... 方法实现省略 ...
    }

    bool _isCjk(String s) {
    // ... 函数实现省略 ...
    }

class _ReadAloudChunk {
    final int paragraphIndex;
    final String speechText;
    final String highlightText;

    const _ReadAloudChunk({
    // ... 函数实现省略 ...
    })

    int get key => paragraphIndex;

class _TtsSegment {
    final String speech;
    final String highlight;

    const _TtsSegment({
    // ... 函数实现省略 ...
    })

enum _TranslationQueueState {

class _TranslationQueueItem {
    final int chapterIndex;
    final int paragraphIndex;
    final String text;

    const _TranslationQueueItem({
    // ... 函数实现省略 ...
    })

    String get key => '$chapterIndex:$paragraphIndex';

class _TranslatedResult {
    final _TranslationQueueItem item;
    final String? translated;
    final bool success;

    const _TranslatedResult({

```

```

    // ... 函数实现省略 ...
}

class _TranslationPageSlice {
    final int chapterIndex;
    final Map<int, String> paragraphsByIndex;

    const _TranslationPageSlice({
    // ... 函数实现省略 ...
})

class _SelectionIllustrationLink {
    final String text;
    final String suffix;

    const _SelectionIllustrationLink({
    // ... 函数实现省略 ...
})

class _ReaderSelectionControls extends MaterialTextSelectionControls {
    final bool isDarkBg;
    final String Function() getSelectedText;
    final VoidCallback onResetSelection;
    final VoidCallback onSuppressReaderTap;
    final Future<void> Function(String text) onReadCurrent;
    final Future<void> Function(String text) onTranslate;
    final Future<void> Function(String text) onExplain;
    final Future<void> Function(String text, String suffix) onIllustration;

    Size getHandleSize(double textLineHeight) {
    // ... 函数实现省略 ...
}

    void _dismissToolbar(TextSelectionDelegate delegate) {
    // ... 函数实现省略 ...
}

    final text = getSelectedText().trim();
    if (text.isEmpty) {
    // ... 方法实现省略 ...
}

    final tp = context.read<TranslationProvider>();
    final readAloud = context.read<ReadAloudProvider>();
    final aiModel = context.read<AiModelProvider>();
    final canReadCurrent = tp.aiReadAloudEnabled && text.isNotEmpty;
    final canExplain = text.isNotEmpty;

    final toolbarFg = isDarkBg ? AppColors.deepSpace : Colors.white;
    final disabledFg = toolbarFg.withOpacityCompat(0.38);
    final baseTheme = Theme.of(context);

    unawaited(() async {
    // ... 方法实现省略 ...
}

```

```

        delegate.copyWithSelection(SelectionChangedCause.toolbar);
        delegate.selectAll(SelectionChangedCause.toolbar);

class ReaderPage extends StatefulWidget {
  final String bookId;

  const ReaderPage({super.key, required this.bookId});
  // ... 函数实现省略 ...
}

class _ReaderPageState extends State<ReaderPage>
  static const int _onlinePrefetchWindow = 5;
  bool _showControls = false;
  bool _isLoading = true;

  String _centerToastText = '';
  bool _txtTocParsing = false;
  int _txtTocParseToken = 0;
  bool _popping = false;

  double _fontSize = 18.0;
  double _lineHeight = 1.4;
  Color _bgColor = const Color(0xFFFF5F9FA); // Default (Day)
  Color _textColor = const Color(0xFF2C3E50);
  bool _followSystemTheme = true;

  bool _aiReadAloudPlaying = false;
  bool _aiReadAloudPreparing = false;
  int _readAloudSession = 0;
  int _readAloudQueuePos = 0;
  int _readAloudAutoSkipRemaining = 0;
  int _readAloudTranslationRevision = -1;
  double _readAloudBackSpinTurns = 0.0;
  double _readAloudForwardSpinTurns = 0.0;
  bool _readAloudFollow = true;
  bool _readAloudNavInFlight = false;
  int _selectionAreaResetToken = 0;

  if (_readAloudPlayerInstance == null) {
  // ... 方法实现省略 ...
}

  final Map<String, Uint8List> _readAloudAudioCache = {};
  final Map<String, Future<Uint8List>> _readAloudAudioInFlight = {};
  final WebSpeechTts _webSpeechTts = createWebSpeechTts();
  bool _currentPageTranslateResumeScheduled = false;
  bool _onlinePrefetchRunning = false;
  bool _onlinePrefetchNeedsRerun = false;
  final Queue<_TranslationQueueItem> _translationQueue = Queue();
  final Queue<_TranslatedResult> _translatedResultsQueue = Queue();
  final Map<String, _TranslationQueueState> _translationQueueStates = {};
  final Set<String> _translationQueueKeys = {};
  int _translationQueueSession = 0;
  bool _translationQueueRunning = false;
  bool _translationInsertRunning = false;
  int _translationQueueTotal = 0;

```

```

int _translationQueueCompleted = 0;
int _translationQueueFailed = 0;
int _translationInsertFailed = 0;
int _translationQueuePendingExternal = 0;
bool _translationQueueInFlight = false;

int _currentChapterIndex = 0;
final Set<int> _expandedChapterIndices = {};

int _currentPageInChapter = 0;
final Map<int, int> _chapterPageCounts = {};

final Map<int, String> _chapterContentCache = {};
final Map<int, String> _chapterPlainText = {};
final Map<int, String> _chapterEffectiveText = {};
final Map<int, List<TextRange>> _chapterPageRanges = {};
final Map<int, String> _chapterPageRangeKeys = {};
final Map<int, String> _chapterFallbackEffectiveText = {};
final Map<int, List<TextRange>> _chapterFallbackPageRanges = {};
final Map<int, String> _chapterFallbackPageRangeKeys = {};
final Map<int, List<TextRange>> _chapterPlainPageRanges = {};
final Map<int, String> _chapterPlainPageRangeKeys = {};
final Map<int, Future<void>> _chapterTextPaginationTasks = {};
final Map<int, String> _chapterTextPaginationTaskKeys = {};
final Map<int, int> _chapterTextPaginationTargetCounts = {};
final Map<int, bool> _chapterTextPaginationComplete = {};
final Map<int, Future<void>> _chapterPlainPaginationTasks = {};
final Map<int, String> _chapterPlainPaginationTaskKeys = {};
final Map<int, int> _chapterPlainPaginationTargetCounts = {};
final Map<int, bool> _chapterPlainPaginationComplete = {};
final Map<int, int> _chapterPaginationLastMs = {};
final Map<int, int> _chapterTitleLength = {};
final Map<int, List<ReaderParagraph>> _chapterParagraphsCache = {};
final Map<int, _ReaderChapter> _chapterPlainLoading = {};
final Map<int, _ReaderChapter> _chapterHtmlLoading = {};

/// Evict cached data for chapters far from the current reading position.
/// Keeps a window of [_kCacheWindow] chapters around [_currentChapterIndex].
static const int _kCacheWindow = 3;

void _evictDistantChapterCaches() {
// ... 函数实现省略 ...
}

double _currentPageProgressInChapter = 0;
bool _relocateAfterTranslationChangeScheduled = false;
bool _readAloudSyncScheduled = false;

late PageController _pageController;
int _pageViewCenterIndex = 1000;

final BookParser _parser = BookParser();
late AnimationController _pulseController;
late AnimationController _readAloudAnimController;
bool _tapMoved = false;

```

```

int _suppressReaderTapUntilMs = 0;
final Map<int, List<_SelectionIllustrationLink>> _selectionIllustrations = {};
final Set<int> _chaptersWithChapterIllustrations = {};
int _lastAiPointsBalance = 0;
bool _lastAiLoaded = false;
bool _lastAiReadAloudEnabled = false;
bool _readAloudFabCollapsed = false;

static const double _kFloatingCapsuleHeight = 52;
static const double _kFloatingHandleWidth = 22;
static const double _kReadAloudExpandedWidth = 150;

double _readAloudFabWidth(bool collapsed) {
// ... 函数实现省略 ...
}

void _collapseFloatingUi() {
// ... 函数实现省略 ...
}

void _setReadAloudFabCollapsed(bool value) {
// ... 函数实现省略 ...
}

void _armFloatingUiAutoHideIfNeeded() {
// ... 函数实现省略 ...
}

void _touchFloatingUi() {
// ... 函数实现省略 ...
}

void _showCenterToast(String msg) {
// ... 函数实现省略 ...
}

void _showTopError(String msg, {bool isError = true}) {
// ... 函数实现省略 ...
}

double _readerTopExtraForOverlay(BuildContext context) {
// ... 函数实现省略 ...
}

void initState() {
// ... 函数实现省略 ...
}

void didChangePlatformBrightness() {
// ... 函数实现省略 ...
}

void didChangeAppLifecycleState(AppLifecycleState state) {
// ... 函数实现省略 ...
}

```

```

    void _startPulseTimer() {
    // ... 函数实现省略 ...
    }

    bool _shouldSkipChapter(String? title, String? href) {
    // ... 函数实现省略 ...
    }

    List<_ReaderChapter> _buildEpubChapters(List<EpubChapterRef> roots) {
    // ... 方法实现省略 ...
    }

    Future<void> _loadSettingsAndBook() async {
    // ... 方法实现省略 ...
    }

    void dispose() {
    // ... 函数实现省略 ...
    }

    final prefs = _prefs!;

// =====
// 文件: presentation/pages/reader/widgets/reader_paragraph.dart
// 原始行数: 14 | 保留行数: 14
// =====

class ReaderParagraph {
    final int index;
    final int start;
    final int end;
    final String text;

    const ReaderParagraph({
        required this.index,
        required this.start,
        required this.end,
        required this.text,
    });
}

// =====
// 文件: ai/translation/translation_types.dart
// 原始行数: 46 | 保留行数: 46
// =====

import 'package:flutter/foundation.dart';

enum TranslationDisplayMode {
    translationOnly,
    bilingual,
}

```

```

@immutable
class TranslationConfig {
  /// Empty means auto-detect (if supported by engine).
  final String sourceLang;

  /// Required.
  final String targetLang;
  final TranslationDisplayMode displayMode;

  const TranslationConfig({
    required this.sourceLang,
    required this.targetLang,
    required this.displayMode,
  });

  TranslationConfig copyWith({
    String? sourceLang,
    String? targetLang,
    TranslationDisplayMode? displayMode,
  }) {
    return TranslationConfig(
      sourceLang: sourceLang ?? this.sourceLang,
      targetLang: targetLang ?? this.targetLang,
      displayMode: displayMode ?? this.displayMode,
    );
  }
}

@immutable
class TranslationResult {
  final String text;
  final bool fromCache;

  const TranslationResult({
    required this.text,
    required this.fromCache,
  });
}

// =====
// 文件: ai/translation/translation_service.dart
// 原始行数: 361 | 保留行数: 149
// =====

import 'dart:async';
import 'dart:collection';

import 'engines/translation_engine.dart';

import 'translation_cache.dart';
import 'translation_queue.dart';
import 'translation_types.dart';

enum TranslationBackend {

```

```

    local,
    online,
}

class TranslationService {
    final TranslationCache cache;
    final void Function(String message, Object? error, StackTrace? st)? logger;

    final TranslationTaskQueue _localQueue =
        TranslationTaskQueue(maxConcurrent: 1);
    final TranslationTaskQueue _onlineQueue =
        TranslationTaskQueue(maxConcurrent: 3);

    final Map<String, Future<String>> _inFlight = {};

    final TranslationEngine engine;
    final TranslationBackend backend;

    /// AI context cache: keep last 3 source paragraphs per (targetLang).

    final Map<String, ListQueue<String>> _aiContextSources = {};

    TranslationService({
        required this.cache,
        this.logger,
        required this.engine,
        required this.backend,
    });

    Duration get _translateTimeout {
        switch (backend) {
            case TranslationBackend.local:
                return const Duration(seconds: 70);
            case TranslationBackend.online:
                return const Duration(seconds: 45);
        }
    }

    TranslationTaskQueue get _queue {
        switch (backend) {
            case TranslationBackend.local:
                return _localQueue;
            case TranslationBackend.online:
                return _onlineQueue;
        }
    }

    String normalizeParagraphText(String text) {
        return text.trim();
    }

    String buildCacheKey({
        required TranslationConfig config,
        required String paragraphText,
    }) {

```

```

final normalized = normalizeParagraphText(paragraphText);
return cache.buildKey(
  engineId: engine.id,
  sourceLang: config.sourceLang,
  targetLang: config.targetLang,
  text: normalized,
);

// =====
// ... ai/translation/translation_service.dart 中间部分代码省略 (217 行) ...
// =====

    return true;
  }

  final stats = _scriptStats(translated);
  final target = config.targetLang.toLowerCase().trim();
  final wantEn = target == 'en' ||
    target.startsWith('en-') ||
    target.contains('english');
  final wantZh = target == 'zh' ||
    target.startsWith('zh-') ||
    target == 'cn' ||
    target.contains('chinese');

  if (wantEn && stats.cjkRatio > 0.22 && stats.latinRatio < 0.18) {
    return true;
  }
  if (wantZh && stats.cjkRatio < 0.12 && stats.latinRatio > 0.22) {
    return true;
  }

  int weird = 0;
  int total = 0;
  for (final r in translated.runes) {
    total++;
    if (r == 0xFFFFD) {
      weird++;
      continue;
    }
    if (r < 0x20 && r != 0x0A && r != 0x09 && r != 0x0D) {
      weird++;
      continue;
    }
  }
  if (total > 0 && weird / total > 0.02) return true;
  return false;
}

_ScriptStats _scriptStats(String s) {
  int total = 0;
  int cjk = 0;
  int latin = 0;
  for (final r in s.runes) {
    total++;

```

```

        if ((r >= 0x4E00 && r <= 0x9FFF) ||
            (r >= 0x3400 && r <= 0x4DBF) ||
            (r >= 0xF900 && r <= 0xFAFF)) {
            cjk++;
            continue;
        }
        if ((r >= 0x41 && r <= 0x5A) || (r >= 0x61 && r <= 0x7A)) {
            latin++;
            continue;
        }
    }
    final t = total <= 0 ? 1 : total;
    return _ScriptStats(
        cjkRatio: cjk / t,
        latinRatio: latin / t,
    );
}
}

class _ScriptStats {
    final double cjkRatio;
    final double latinRatio;

    const _ScriptStats({
        required this.cjkRatio,
        required this.latinRatio,
    });
}

// =====
// 文件: presentation/providers/translation_provider.dart
// 原始行数: 1561 | 保留行数: 250
// =====

import 'dart:async';
import 'dart:convert';

import 'package:flutter/foundation.dart';
import 'package:flutter/services.dart';
import 'package:http/http.dart' as http;

import 'package:shared_preferences/shared_preferences.dart';
import '../ai/hunyuan/hunyuan_translation_engine.dart';
import '../ai/config/auth_service.dart';
import '../ai/tencentcloud/embedded_public_hunyuan_credentials.dart';
import '../ai/tencentcloud/tencent_api_client.dart';
import '../ai/tencentcloud/tencent_cloud_exception.dart';
import '../ai/tencentcloud/tmt_translation_engine.dart';
import '../ai/translation/engines/translation_engine.dart';
import '../ai/translation/translation_cache.dart';
import '../ai/translation/translation_service.dart';
import '../ai/translation/translation_types.dart';
import 'ai_model_provider.dart';

```

```

enum TranslationMode {

enum ReadAloudEngine {

class AzureTranslationEngine implements TranslationEngine {
    String.fromEnvironment('AIRREAD_AZURE_TRANSLATOR_KEY', defaultValue: '');

    final http.Client _client;
    final String endpoint;
    final String key;
    final String region;

    static bool get isConfigured => _defaultEndpoint.trim().isNotEmpty;

    bool get isUsable => endpoint.isNotEmpty;

    String get id => 'azure_translator';

    final normalized = text.trim();
    if (!isUsable) {
// ... 方法实现省略 ...
}

    final from = _normalizeAzureLang(sourceLang, allowEmpty: true);
    final to = _normalizeAzureLang(targetLang);
    if (to.isEmpty) {
// ... 方法实现省略 ...
}

    final uri = _buildUri(from: from, to: to);
    if (key.isNotEmpty) {
// ... 方法实现省略 ...
}

    if (resp.statusCode < 200 || resp.statusCode >= 300) {
// ... 方法实现省略 ...
}

    final decoded = jsonDecode(utf8.decode(resp.bodyBytes));
    if (decoded is! List || decoded.isEmpty) {
// ... 方法实现省略 ...
}

    final first = decoded.first;
    if (first is! Map) {
// ... 方法实现省略 ...
}

    final detectedLanguage = first['detectedLanguage'];
    final translations = first['translations'];
    final item = translations.first;
    final out = item['text']?.toString() ?? '';
    detectedNormalized.toLowerCase() == to.toLowerCase() {
        return normalized;
    }
    return out;

    final out = <String>[];

```

```

        out.add(await translate(
return out;

Uri _buildUri({required String from, required String to}) {
// ... 函数实现省略 ...
}

    final now = DateTime.now();
    final cached = _edgeTokenCache;
    final uri = Uri.parse('https://edge.microsoft.com/translate/auth');
    throw StateError('Azure token HTTP ${resp.statusCode}');
// ... 函数实现省略 ...
}

    final token = resp.body.trim();
    final expiresAt = now.add(const Duration(minutes: 8));
    return token;

String _normalizeAzureLang(String lang, {bool allowEmpty = false}) {
// ... 函数实现省略 ...
}

class _AzureTokenCache {
    final String token;
    final DateTime expiresAt;
    const _AzureTokenCache({required this.token, required this.expiresAt});
// ... 函数实现省略 ...
}

class FallbackTranslationEngine implements TranslationEngine {
    final TranslationEngine primary;
    final TranslationEngine fallback;
    final bool Function()? primaryAvailable;
    final bool Function()? fallbackAvailable;

    String get id => '${primary.id}_fallback_${fallback.id}';

    final usePrimary = primaryAvailable?.call() ?? true;

    final usePrimary = primaryAvailable?.call() ?? true;

class TranslationProvider extends ChangeNotifier {
    static const _kCfgFrom = 'tr_cfg_from';
    static const _kCfgTo = 'tr_cfg_to';
    static const _kCfgMode = 'tr_cfg_mode';
    static const _kTranslationMode = 'tr_translation_mode';
    static const _kAiTranslateEnabled = 'tr_ai_translate_enabled';
    static const _kAiReadAloudEnabled = 'tr_ai_read_aloud_enabled';
    static const _kTtsVoiceType = 'tr_tts_voice_type';
    static const _kTtsSpeed = 'tr_tts_speed';
    static const _kTtsSpeedLevel = 'tr_tts_speed_level';
    static const _kReadAloudEngine = 'tr_read_aloud_engine';
    static const _kReadTranslationEnabled = 'tr_read_translation_enabled';

    static const _kUserTencentKeysEnabled = 'user_tencent_keys_enabled';

```

```

late TranslationService _service;
late TranslationService _machineService;
String _machineEngineId = '';
String _bigModelEngineId = '';

TranslationMode _translationMode = TranslationMode.machine;
ReadAloudEngine _readAloudEngine = ReadAloudEngine.local;
bool _aiTranslateEnabledGlobal = false;
bool _aiReadAloudEnabledGlobal = false;

bool _aiTranslateEnabled = false;
bool _aiReadAloudEnabled = false;
bool _readTranslationEnabled = false;
bool _loaded = false;

int _ttsVoiceType = 601003;
double _ttsSpeed = 0.0;
bool _usingPersonalTencentKeys = false;
bool _localReadAloudAvailable = true;

bool _notifyScheduled = false;
int _cacheRevision = 0;
int _lastPointsInsufficientToastMs = 0;

void updateAiModel(AiModelProvider model) {
    // ... 函数实现省略 ...
}

int get pointsBalance => _aiModel?.pointsBalance ?? 0;

int _estimateOnlineTranslateCost(String text) {
    // ... 函数实现省略 ...
}

final ai = _aiModel;
final override = ai.debugPointsOverride;
final next = (override - cost);

void debugConsumeOnlinePoints(int cost) {
    // ... 函数实现省略 ...
}

bool _onlineTranslateEntitled() {
    // ... 函数实现省略 ...
}

bool _onlineTranslateEnoughForText(String text) {
    // ... 函数实现省略 ...
}

void _onAiModelChanged() {
    // ... 函数实现省略 ...
}

void _syncFeatureFlagsToModel() {

```

```

// ... 函数实现省略 ...
}

void _rebuildService() {
// ... 函数实现省略 ...
}

bool get loaded => _loaded;

TranslationConfig get config => _config;
bool get applyToReader => _aiTranslateEnabled;
bool get aiTranslateEnabled => _aiTranslateEnabled;
bool get aiReadAloudEnabled => _aiReadAloudEnabled;
bool get readTranslationEnabled => _readTranslationEnabled;
int get cacheRevision => _cacheRevision;
TranslationMode get translationMode => _translationMode;
ReadAloudEngine get readAloudEngine => _readAloudEngine;
int get ttsVoiceType => _ttsVoiceType;
double get ttsSpeed => _ttsSpeed;
bool get usingPersonalTencentKeys => _usingPersonalTencentKeys;
bool get localReadAloudAvailable => _localReadAloudAvailable;

void dispose() {
// ... 函数实现省略 ...
}

void _scheduleNotify([Duration delay = const Duration(milliseconds: 50)]) {
// ... 函数实现省略 ...
}

final prefs = await SharedPreferences.getInstance();

final mode = prefs.getString(_kCfgMode);
final trMode = prefs.getString(_kTranslationMode);

final bookId = _boundBookId;
prefs.getBool(_bookKey(_kAiTranslateEnabled, bookId)) ??
prefs.getBool(_bookKey(_kAiReadAloudEnabled, bookId)) ??

double oldSpeed = prefs.getDouble(_kTtsSpeed) ?? 1.0;

final engine = prefs.getString(_kReadAloudEngine);

final from = prefs.getString(_kCfgFrom) ?? _config.sourceLang;
final to = prefs.getString(_kCfgTo) ?? _config.targetLang;

TranslationDisplayMode displayMode = _config.displayMode;

_config.copyWith(

var v = lang.trim();
return mode == TranslationMode.bigModel ? 'zh-TR' : 'zh-TW';
return v;

final s = source.isEmpty || _bigModelLangs.contains(source) ? source : '';

```

```

final s = _machineSourceLangs.contains(source) ? source : '';
final allowed = _machineTargetsBySource[s] ?? _machineTargetsBySource['']!;

bool ok = true;

final prefs = await SharedPreferences.getInstance();
final bookId = _boundBookId;

static String _bookKey(String baseKey, String bookId) {

// =====
// 文件: presentation/pages/reader/widgets/translation_sheet.dart
// 原始行数: 258 | 保留行数: 101
// =====

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../../../../../core/theme/app_colors.dart';
import '../../../../../core/theme/app_tokens.dart';
import '../../../../../widgets/glass_panel.dart';
import '../../../../../providers/translation_provider.dart';
import '../../../../../ai/translation/translation_types.dart';

/// Translation settings sheet.
///
/// Note: In this app the "apply translation to reader" switch is controlled from
/// the AI companion main panel. This sheet focuses on configuration.
class TranslationSheet extends StatelessWidget {
  final Color bgColor;
  final Color textColor;

  const TranslationSheet({
    super.key,
    required this.bgColor,
    required this.textColor,
  });

  static const _langs = <String, String>{
    '': '自动',
    'zh-Hans': '中文',
    'en': '英语',
    'ja': '日语',
    'ko': '韩语',
    'fr': '法语',
    'de': '德语',
    'es': '西班牙语',
    'ru': '俄语',
  };

  @override
  Widget build(BuildContext context) {
    final provider = context.watch<TranslationProvider>();
    final cfg = provider.config;

```

```

final panelBg = bgColor;
final panelText = textColor;

return GlassPanel.sheet(
  surfaceColor: panelBg,
  opacity: AppTokens.glassOpacityDense,
  child: SafeArea(
    top: false,

// =====
// ... presentation/pages/reader/widgets/translation_sheet.dart 中间部分代码省略 (162 行) ...
// =====

  );
}

static Widget _dropdown({
  required String label,
  required String value,
  required Map<String, String> items,
  required ValueChanged<String?> onChanged,
  required Color textColor,
  required Color dropdownColor,
}) {
  return Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Text(label,
        style: TextStyle(color: textColor, fontWeight: FontWeight.w600)),
      const SizedBox(height: 6),
      SizedBox(
        height: 50,
        child: DropdownButtonFormField<String>(
          key: ValueKey('$label|$value'),
          initialValue: items.containsKey(value) ? value : items.keys.first,
          dropdownColor: dropdownColor,
          decoration: InputDecoration(
            isDense: true,
            filled: true,
            fillColor: Colors.white,
            contentPadding:
              const EdgeInsets.symmetric(horizontal: 12, vertical: 12),
            border:
              OutlineInputBorder(borderRadius: BorderRadius.circular(12)),
          ),
          items: items.entries
            .map(
              (e) => DropdownMenuItem<String>(
                value: e.key,
                child: Text(e.value, style: const TextStyle(fontSize: 13)),
              ),
            )
            .toList(),
          onChanged: onChanged,

```

```

        ),
    ),
],
);
}
}

// =====
// 文件: ai/tencent_tts/tencent_tts_client.dart
// 原始行数: 298 | 保留行数: 149
// =====

import 'dart:async';
import 'dart:convert';
import 'dart:math';
import 'dart:typed_data';

import 'package:crypto/crypto.dart';

import '../tencentcloud/tencent_api_client.dart';
import '../tencentcloud/tencent_credentials.dart';
import '../tencentcloud/embedded_public_hunyuan_credentials.dart';
import 'tts_ws.dart';

class TencentTtsResult {
    final String audioBase64;
    final String requestId;

    const TencentTtsResult({
        required this.audioBase64,
        required this.requestId,
    });
}

class TencentTtsClient {
    static const String _host = 'tts.tencentcloudapi.com';
    static const String _service = 'tts';
    static const String _version = '2019-08-23';

    static const String _streamHost = 'tts.cloud.tencent.com';
    static const String _streamPath = '/stream_wsv2';

    final TencentApiClient _api;
    final TencentCredentials _credentials;

    TencentTtsClient({
        TencentApiClient? api,
        required TencentCredentials credentials,
    }) : _api = api ?? TencentApiClient(),
        _credentials = credentials;

    Future<TencentTtsResult> textToVoice({
        required String text,
        String codec = 'mp3',
    }) {

```

```

        int? voiceType,
        double? speed,
    }) async {
        final sessionId = _randomSessionId();
        final payload = <String, dynamic>{
            'Text': text,
            'SessionId': sessionId,
            'Codec': codec,
        };
        if (voiceType != null) payload['VoiceType'] = voiceType;
        if (speed != null) payload['Speed'] = speed;

        final resp = await _api.postJson(
            host: _host,
            service: _service,
            action: 'TextToVoice',
            version: _version,
            secretId: _credentials.secretId,
            secretKey: _credentials.secretKey,
            useProxy: !usingPersonalTencentKeys(),
            payload: payload,
            timeout: const Duration(seconds: 30),
        );

        return TencentTtsResult(
            audioBase64: resp['Audio']?.toString() ?? '',
            requestId: resp['RequestId']?.toString() ?? '',
        );
    }
}

```

```

// =====
// ... ai/tencent_tts/tencent_tts_client.dart 中间部分代码省略 (154 行) ...
// =====

```

```

    } finally {
        final wd = watchdog;
        if (wd != null) wd.cancel();
        try {
            final socket = ws;
            if (socket != null) {
                await socket.close();
            }
        } catch (_) {}
    }
}

```

```

String _randomSessionId() {
    final r = Random();
    final t = DateTime.now().millisecondsSinceEpoch.toString();
    final high = r.nextInt(1 << 16);
    final low = r.nextInt(1 << 16);
    final n = ((high << 16) | low).toRadixString(16);
    return 'airread-$t-$n';
}

```

```

String _buildStreamWsv2Url({
  required int appId,
  required String secretId,
  required String secretKey,
  required String sessionId,
  required String codec,
  int? voiceType,
  double? speed,
  int sampleRate = 16000,
  bool enableSubtitle = false,
}) {
  final timestamp = DateTime.now().millisecondsSinceEpoch ~/ 1000;
  final expired = timestamp + 24 * 60 * 60;

  final params = <String, String>{
    'Action': 'TextToStreamAudioWsv2',
    'AppId': appId.toString(),
    'Codec': codec,
    'EnableSubtitle': enableSubtitle ? 'True' : 'False',
    'Expired': expired.toString(),
    'SampleRate': sampleRate.toString(),
    'SecretId': secretId,
    'SessionId': sessionId,
    'Timestamp': timestamp.toString(),
    'Volume': '0',
  };

  if (voiceType != null) {
    params['VoiceType'] = voiceType.toString();
  }
  if (speed != null) {
    params['Speed'] = speed.toString();
  }

  final keys = params.keys.toList()..sort();
  final canonicalQuery = keys
    .map((k) =>
      '${Uri.encodeQueryComponent(k)}=${Uri.encodeQueryComponent(params[k] ?? '')}')
    .join('&');

  final signText = 'GET$_streamHost$_streamPath?$canonicalQuery';
  final hmacSha1 = Hmac(sha1, utf8.encode(secretKey));
  final signature =
    base64Encode(hmacSha1.convert(utf8.encode(signText)).bytes);

  final fullQuery =
    '$canonicalQuery&Signature=${Uri.encodeQueryComponent(signature)}';
  return 'wss://$_streamHost$_streamPath?$fullQuery';
}
}

```

```
// 原始行数: 1419 | 保留行数: 250
// =====

import 'dart:async';
import 'dart:convert';
import 'dart:io';

import 'package:audioplayers/audioplayers.dart';
import 'package:flutter/foundation.dart';
import 'package:flutter/services.dart';
import 'package:path_provider/path_provider.dart';
import 'package:shared_preferences/shared_preferences.dart';

import '../..ai/tencent_tts/tencent_tts_client.dart';
import '../..ai/tencentcloud/tencent_cloud_exception.dart';
import '../..ai/tencentcloud/embedded_public_hunyuan_credentials.dart';
import '../..ai/translation/translation_types.dart';
import '../pages/reader/tts_web_speech.dart';
import '../pages/reader/widgets/reader_paragraph.dart';
import 'translation_provider.dart';

class ReadAloudPosition {
  final String bookId;
  final int chapterIndex;
  final int paragraphIndex;
  final int chunkIndexInParagraph;
  final int highlightOffsetInParagraph;
  final int chapterTextOffset;
  final String highlightText;

  const ReadAloudPosition({
    // ... 函数实现省略 ...
  })

  final bookId = (value['bookId'] ?? '').toString();
  final chapterIndex = _asInt(value['chapterIndex']);
  final paragraphIndex = _asInt(value['paragraphIndex']);
  final chunkIndexInParagraph = _asInt(value['chunkIndexInParagraph']);
  final chapterTextOffset = _asInt(value['chapterTextOffset']) ?? 0;
  final highlightText = (value['highlightText'] ?? '').toString();
  return null;

  return switch (v) {
    // ... 函数实现省略 ...
  }
}

class ReadAloudProvider extends ChangeNotifier {

  bool _queueDirty = false;

  if (_playerInstance == null) {
    // ... 方法实现省略 ...
  }

  final WebSpeechTts _webSpeechTts = createWebSpeechTts();
  bool _localSpeakingPollInFlight = false;

```

```

int _localSpeakingPollSession = 0;
bool _localSpeakingPollEverSpeaking = false;
int _localSpeakingPollErrorCount = 0;

final Map<String, Uint8List> _onlineAudioCache = {};
final Map<String, Future<Uint8List>> _onlineAudioInFlight = {};
final int _onlinePrefetchDistance = 3;
bool _onlineTransitioning = false;

bool _initialized = false;
bool _playing = false;
bool _preparing = false;
bool _paused = false;
bool _endedNaturally = false;
int _session = 0;

int _queuePos = 0;

bool _isPointsInsufficientError(Object e) {
// ... 函数实现省略 ...
}

bool get playing => _playing;
bool get preparing => _preparing;
bool get paused => _paused;
bool get endedNaturally => _endedNaturally;

void updateTranslationProvider(TranslationProvider tp) {
// ... 函数实现省略 ...
}

Future<void> _ensureInit() async {
// ... 方法实现省略 ...
}

void _onTtsConfigChanged() {
// ... 函数实现省略 ...
}

void _restoreLastPosition() {
// ... 函数实现省略 ...
}

Future<void> _persistPosition(ReadAloudPosition p) async {
// ... 方法实现省略 ...
}

void attachChapter({
// ... 函数实现省略 ...
}

bool get isActiveForBook =>

bool get atQueueStart => _queue.isEmpty || _queuePos <= 0;
    
```

```

bool get atQueueEnd =>

    final tp = _tp;

    final queue = _buildQueue(paragraphs: paragraphs, startParagraphIndex: 0);

    return true;

    final tp = _tp;

    if (_playing && _bookId == bookId && _chapterIndex == chapterIndex) {
// ... 方法实现省略 ...
    }

    if (_paused && _bookId == bookId && _chapterIndex == chapterIndex) {
// ... 方法实现省略 ...
    }

    final savedPosition = _position;
        startPara.clamp(0, paragraphs.isEmpty ? 0 : paragraphs.length - 1);

    final queue = _buildQueue(paragraphs: paragraphs, startParagraphIndex: 0);

    final session = ++_session;

    return true;

Future<void> pause() async {
// ... 方法实现省略 ...
}

    final session = ++_session;

    final prev = (_queuePos - 1).clamp(0, _queue.length - 1);
        return true;
    final session = ++_session;
    return true;

    final next = (_queuePos + 1).clamp(0, _queue.length - 1);
        return true;
    final session = ++_session;
    return true;

    final tp = _tp;

    final queue = _buildQueue(paragraphs: paragraphs, startParagraphIndex: 0);
        return true;
    final session = ++_session;
    return true;

    final tp = _tp;

    final queue = _buildQueue(paragraphs: paragraphs, startParagraphIndex: 0);
        return true;
    final session = ++_session;

```

```

return true;

final entry = _queue[_queuePos];
int highlightOffsetInParagraph = 0;
    final h = entry.highlightText.trim();
    final idx = para.text.indexOf(h);

_onlineAudioInFlight.clear();

final path = _tempFilePath;
    final f = File(path);

final pos = _position;
final wasPlaying = _playing || _preparing;

final tp = _tp;
final out = <_ReadAloudChunk>[];
    final p = paragraphs[i];
    final segments = _splitTtsSegments(speechBase);
    final s = segments[seg];
    final speech = _normalizeSpeechText(s.speech).trim();
    final highlight = s.highlight.trim();
    out.add(
return out;

int _findQueueStartPos({
// ... 函数实现省略 ...
}

final tp = _tp;

    final old = _queue[_queuePos];

final entry = _queue[_queuePos];
int highlightOffsetInParagraph = 0;
    final h = entry.highlightText.trim();
    final idx = para.text.indexOf(h);

        final need = entry.speechText.trim().length * 10;
        tp.debugConsumeOnlinePoints(need);
        final dir = await getTemporaryDirectory();

final tp = _tp;

void _prefetchOnlineAhead({
// ... 函数实现省略 ...
}

void _onOnlineChunkDone() {
// ... 函数实现省略 ...
}

void _onLocalChunkDone() {
// ... 函数实现省略 ...
}

```

```

    void _onLocalChunkError(String message) {
    // ... 函数实现省略 ...
    }

    void _stopLocalSpeakingPoll() {
    // ... 函数实现省略 ...
    }

    void _startLocalSpeakingPoll({
    // ... 函数实现省略 ...
    })

    String _normalizeSpeechText(String text) {
    // ... 函数实现省略 ...
    }

    String _speechTextForParagraph({
    // ... 函数实现省略 ...
    })

    bool _isSkippableWhitespaceCu(int cu) {
    // ... 函数实现省略 ...
    }

    int _ttsUnitCount(String s) {
    // ... 函数实现省略 ...
    }

    final base = text.trim();
    final sentenceRegex = RegExp(r'^[■.!?;,]+[■.!?;,]*\s*');
    final out = <_TtsSegment>[];
    final buffer = StringBuffer();

    // =====
    // 文件: ai/reading/qa_service.dart
    // 原始行数: 364 | 保留行数: 149
    // =====

    import 'dart:async';
    import 'dart:ui'; // for TextRange

    import '../hunyuan/hunyuan_text_client.dart';
    import '../local_llm/llm_client.dart';
    import '../tencentcloud/tencent_credentials.dart';

    String _getCurrentDateInfo() {
        final now = DateTime.now();
        final weekday = ['一', '二', '三', '四', '五', '六', '日'][now.weekday - 1];
        return '${now.year}年${now.month}月${now.day}日（周$weekday）';
    }

    class ReadingContextService {
        final Map<int, String> chapterContentCache;
        final int currentChapterIndex;
    }

```

```

final int currentPageInChapter;
final Map<int, List<TextRange>> chapterPageRanges;

ReadingContextService({
    required this.chapterContentCache,
    required this.currentChapterIndex,
    required this.currentPageInChapter,
    required this.chapterPageRanges,
});

String getCurrentChapterContent() {
    final chapterContent = chapterContentCache[currentChapterIndex] ?? '';
    if (chapterContent.isEmpty) return '';

    // 简单清理 XML 标签等
    final cleaned = _cleanContent(chapterContent);
    return cleaned;
}

String _cleanContent(String content) {
    // 简单移除 HTML/XML 标签
    return content.replaceAll(RegExp(r'<[^>]*>'), '').trim();
}

// QA类型枚举
enum QAType {
    general,
    summary,
    keyPoints,
}

/// QA流式输出块
class QStreamChunk {
    final String content;
    final String? reasoningContent;
    final bool isReasoning;
    final bool isComplete;

    QStreamChunk({
        required this.content,
        this.reasoningContent,
        this.isReasoning = false,
        this.isComplete = false,
    });
}

String buildOnlineQaPrompt({
    required ReadingContextService contextService,
    required String question,
    required QAType qaType,
    String? history,
}) {
    String prompt;

```

```

switch (qaType) {

// =====
// ... ai/reading/qa_service.dart 中间部分代码省略 (220 行) ...
// =====

    );
  }
}

Stream<QAStramChunk> _askLocalModel(
  String question,
  QAType qaType, {
  String? history,
}) async* {
  // 使用适合平台的本地 LLM 客户端
  final client = createLocalLlmClient();
  final initialized = await client.initialize(model: localModelId);

  if (!initialized) {
    yield QAStramChunk(
      content: '本地模型初始化失败, 请检查模型文件是否正确放置。',
      isComplete: true,
    );
    return;
  }

  final prompt = buildLocalQaPrompt(
    contextService: contextService,
    question: question,
    qaType: qaType,
    history: history,
  );

  // MNN 使用固定的上下文大小
  const maxCtx = 4096;
  final caps = _computeLocalCaps(maxCtx);

  await for (final delta in client.generateStream(
    prompt: prompt,
    maxTokens: caps.maxNewTokens,
  )) {
    if (delta.isEmpty) continue;
    yield QAStramChunk(content: delta);
  }

  await client.dispose();
}

_LocalCaps _computeLocalCaps(int? maxContextTokens) {
  final contextSize = maxContextTokens ?? 4096;
  final usable = contextSize - _localQaContextReserveTokens;

  // 分配策略: 调整为 1/2 给输出, 以支持长文本生成 (如插图分析)
  // 之前是 1/3, 对于 4096 窗口, 输出约 1194, 现在提升到约 1792

```

```

        int maxNew = (usable ~/ 2).clamp(256, _localQaHardMaxNewTokens);
        int maxInput = (usable - maxNew).clamp(256, _localQaHardMaxInputTokens);

        return _LocalCaps(maxInputTokens: maxInput, maxNewTokens: maxNew);
    }
}

String _tailText(String input, int maxChars) {
    final s = input.trim();
    if (s.length <= maxChars) return s;
    return s.substring(s.length - maxChars);
}

String _squashSpaces(String input) {
    return input.replaceAll(RegExp(r'\s+'), ' ').trim();
}

class _LocalCaps {
    final int maxInputTokens;
    final int maxNewTokens;
    _LocalCaps({required this.maxInputTokens, required this.maxNewTokens});
}

// =====
// 文件: ai/illustration/illustration_service.dart
// 原始行数: 723 | 保留行数: 180
// =====

import 'dart:async';
import 'dart:io';
import 'package:flutter/foundation.dart';
import 'package:http/http.dart' as http;
import 'package:path/path.dart' as path;
import 'package:uuid/uuid.dart';

import '../hunyuan/hunyuan_image_client.dart';
import '../hunyuan/hunyuan_text_client.dart';
import '../tencentcloud/tencent_credentials.dart';
import 'illustration_item.dart';

typedef _AnchorRange = ({

class IllustrationService {
    final HunyuanImageClient _imageClient;
    final HunyuanTextClient _textClient;
    final String _baseStoragePath;

    static const Duration _pollInterval = Duration(seconds: 3);
    static const int _maxPollCount = 40;

    int effectiveCount = count;
    if (effectiveCount <= 0) {
        // ... 方法实现省略 ...
    }
}

```

```

        if (effectiveCount > paragraphs.length) {
// ... 方法实现省略 ...
}

        if (useLocalModel) {
// ... 方法实现省略 ...
}

        final items = <IllustrationItem>[];

        for (int i = 0; i < anchors.length; i++) {
// ... 方法实现省略 ...
}

        return items;

        final chunks = (count / 6).ceil().clamp(1, 3).clamp(1, paragraphs.length);
        final actualChunks = chunkAnchors.length;

        final weights = <int>[];
        final caps = <int>[];
        for (final a in chunkAnchors) {
// ... 方法实现省略 ...
}

        final items = <IllustrationItem>[];
        int globalIndex = 0;

        for (int i = 0; i < actualChunks; i++) {
// ... 方法实现省略 ...
}

        return items;

String _buildSceneExtractionPrompt({
// ... 函数实现省略 ...
}

String _buildRepairPrompt({
// ... 函数实现省略 ...
}

        final clean = _stripModelNoise(raw);
        final lines = clean.split('\n');
        final out = <String>[];
        final re = RegExp(r'^\s*(\d+)[\.\■\s]\s*(.*)$');

        final t = line.trim();
        final m = re.firstMatch(t);
        final idx = int.tryParse(m.group(1) ?? '');
        final content = _normalizeModelPrompt(m.group(2) ?? '');
        out.add(content);

        final fallback = <String>[];
        final t = _normalizeModelPrompt(line);
        fallback.add(t);

```

```

final combined = <String>[...out, ...fallback];

final n = weights.length;

final out = List<int>.filled(n, 0);
int sumCaps = 0;
final target = total.clamp(0, sumCaps);

int baseSum = 0;
    final cap = caps[i] < 0 ? 0 : caps[i];
    final base = minEach.clamp(0, cap);

int remaining = target - baseSum;
    int over = -remaining;
    final reducible = out[i];
    final dec = reducible < over ? reducible : over;
    return out;

int sumW = 0;
final desired = List<double>.filled(n, 0);
    final w = weights[i] <= 0 ? 1 : weights[i];

final frac = <({int idx, double frac})>[];
int used = 0;
    final available = (caps[i] - out[i]).clamp(0, 1 << 30);
    int add = desired[i].floor();
    final f = desired[i] - desired[i].floor();
    frac.add((idx: i, frac: f));

int left = remaining - used;
frac.sort((a, b) => b.frac.compareTo(a.frac));
int guard = 0;
    bool advanced = false;
    final idx = frac[i].idx;

return out;

final out = <String>[];
final seen = <String>{};
    final t = _normalizeModelPrompt(s);
    final key = t.replaceAll(RegExp(r'[■,■\.\■■;■:\s]+'), '').toLowerCase();
    seen.add(key);
    out.add(t);
return out;

final p = _normalizeModelPrompt(prompt);

int count = 0;
    final status = await _imageClient.queryTextToImageJob(jobId);
    final code = status['JobStatusCode'];

    final urls = status['ResultImage'];
    final url = urls.first.toString();
    
```

```

        final resp = await http.get(Uri.parse(url));
        throw Exception('Failed to download image: ${resp.statusCode}');
    // ... 函数实现省略 ...
}

    final dir = Directory(path.join(_baseStoragePath, 'illustration'));

    final filePath = path.join(dir.path, '$jobId.jpg');
    final file = File(filePath);
    return filePath;

    final sw = Stopwatch()..start();
    final buffer = StringBuffer();
    await for (final chunk in stream) {
    // ... 函数实现省略 ...
}

    final out = buffer.toString();
    return out;
return out;

    final cap = count.clamp(1, paragraphs.length);

    int totalLen = 0;
    final lengths = <int>[];
    final len = p.length;
    lengths.add(len);

    final targetLenPerChunk = totalLen / cap;
    final out = <_AnchorRange>[];

    int currentStart = 0;
    int currentLen = 0;

    out.add((anchorStart: currentStart, anchorEnd: paragraphs.length - 1));

    int end = currentStart;

    final len = lengths[end];

    final diffCurrent = (targetLenPerChunk - currentLen).abs();
    final diffNext = ((currentLen + len) - targetLenPerChunk).abs();

    final realEnd = (end - 1).clamp(currentStart, paragraphs.length - 1);
    out.add((anchorStart: currentStart, anchorEnd: realEnd));

    // =====
    // 文件: presentation/widgets/illustration_panel.dart
    // 原始行数: 1399 | 保留行数: 200
    // =====

import 'dart:async';
import 'dart:io';
import 'dart:math' as math;
import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';

```

```
import 'package:provider/provider.dart';
import 'package:shared_preferences/shared_preferences.dart';
import '../ai/illustration/illustration_item.dart' as ill;
import '../ai/tencentcloud/tencent_cloud_exception.dart';
import '../ai/config/auth_service.dart';
import '../ai/tencentcloud/embedded_public_hunyuan_credentials.dart';
import '../ai/local_llm/model_manager.dart';
import '../core/theme/app_colors.dart';
import '../core/theme/app_tokens.dart';
import '../models/ai_chat_model_choice.dart';
import '../providers/ai_model_provider.dart';
import '../providers/illustration_provider.dart';
import '../providers/translation_provider.dart';
import 'scene_image.dart';
import 'points_wallet.dart';
import 'ai_inference_top_row.dart';
```

```
class IllustrationPanel extends StatefulWidget {
  final bool isDark;
  final Color bgColor;
  final Color textColor;
  final String bookId;
  final Map<int, String> chapterTextCache;
  final int currentChapterIndex;
  final String? chapterIdSuffix;
  final ValueChanged<int>? onChapterIllustrationsGenerated;
  final void Function(String, {bool isError})? onShowTopMessage;
  // ... 函数实现省略 ...
}
```

```
class _IllustrationPanelState extends State<IllustrationPanel> {
  static const int _imageCostPoints = 20000;

  String _styleKey = '国风';
  String _ratioKey = '1:1';
  AiChatModelChoice _modelChoice = AiChatModelChoice.onlineHunyuan;
  bool _thinkingEnabled = true;
  final Set<String> _pendingGenerateIds = {};

  String _toastText = '';
  bool _toastIsError = true;

  String _panelPopupText = '';

  void initState() {
    // ... 函数实现省略 ...
  }

  Future<void> _loadPrefs() async {
    // ... 方法实现省略 ...
  }

  Future<void> _setModelChoice(AiChatModelChoice value) async {
    // ... 方法实现省略 ...
  }
}
```

```

    Future<void> _setThinkingEnabled(bool value) async {
// ... 方法实现省略 ...
}

    void _showToast(String msg, {bool isError = true}) {
// ... 函数实现省略 ...
}

    void _showPanelPopup(String msg) {
// ... 函数实现省略 ...
}

    String _friendlyErrorMessage(Object e) {
// ... 函数实现省略 ...
}

    double _ratioKeyToAspect(String k) {
// ... 函数实现省略 ...
}

    String _chapterId() {
// ... 函数实现省略 ...
}

    String _chapterText() {
// ... 函数实现省略 ...
}

    Future<bool> _confirmRegenerate() async {
// ... 方法实现省略 ...
}

    final content = _chapterText();
    if (content.isEmpty) {
// ... 方法实现省略 ...
}

    final onlineEntitled = aiModel.pointsBalance > 0 || usingPersonal;

    bool thinkingForOnline = false;
    final modelKey = _modelChoice.name;
    final thinkingSupported = _modelChoice != AiChatModelChoice.localMiniCpm05b;

    if (_modelChoice.isLocal) {
// ... 方法实现省略 ...
}

    final chapterId = _chapterId();
    final stylePrefix = _stylePrompts[_styleKey] ?? _stylePrompts['国风'];

    _pendingGenerateIds.add(item.id);

    provider.markImageCharged(
        _pendingGenerateIds.remove(item.id);

```

```

    void dispose() {
    // ... 函数实现省略 ...
    }

    Widget build(BuildContext context) {
    // ... 函数实现省略 ...
    }

    Widget _settingsArea({
    // ... 函数实现省略 ...
    })

    Widget _stepperRow({
    // ... 函数实现省略 ...
    })

    class _IllustrationList extends StatelessWidget {
        final List<ill.IllustrationItem> items;
        final Color cardBg;
        final bool canGenerate;
        final double aspect;
        final Future<void> Function(ill.IllustrationItem item) onGenerate;

        const _IllustrationList({
        // ... 函数实现省略 ...
        })

        Widget build(BuildContext context) {
        // ... 函数实现省略 ...
        }

        class _IllustrationCard extends StatelessWidget {
            final ill.IllustrationItem item;
            final Color cardBg;
            final bool canGenerate;
            final double aspect;
            final Future<void> Function() onGenerate;
            final Future<void> Function(String prompt) onUpdatePrompt;

            const _IllustrationCard({
            // ... 函数实现省略 ...
            })

            final controller = TextEditingController(text: initial);
            final v = saved.trim();

            Positioned.fill(

            Widget build(BuildContext context) {
            // ... 函数实现省略 ...
            }

            final p = item.localImagePath;

            class _BrushLoading extends StatefulWidget {

```

```

const _BrushLoading();

class _BrushLoadingState extends State<_BrushLoading>
  late final AnimationController _controller;

  void initState() {
    // ... 函数实现省略 ...
  }

  void dispose() {
    // ... 函数实现省略 ...
  }

  Widget build(BuildContext context) {
    // ... 函数实现省略 ...
  }

class _PromptWithEditIcon extends StatelessWidget {
  final String text;
  final Color textColor;
  final int maxLines;
  final VoidCallback onEdit;

  const _PromptWithEditIcon({
    // ... 函数实现省略 ...
  })

  Widget build(BuildContext context) {
    // ... 函数实现省略 ...
  }
}

// =====
// 文件: presentation/providers/ai_model_provider.dart
// 原始行数: 540 | 保留行数: 120
// =====

import 'dart:async';
import 'dart:convert';
import 'package:flutter/foundation.dart';
import 'package:http/http.dart' as http;
import 'package:shared_preferences/shared_preferences.dart';
import '../ai/local_llm/llm_client.dart';
import '../ai/local_llm/model_manager.dart';
import '../ai/local_llm/mnn_model_downloader.dart';
import '../ai/local_llm/mnn_model_spec.dart';
import '../ai/config/auth_service.dart';
import '../ai/tencentcloud/tencent_api_client.dart';

enum ModelInstallStatus {

class AiModelProvider extends ChangeNotifier {
  static const String _kPointsBalance = 'points_balance';
  static const String _kDebugPointsOverride = 'debug_points_override';
  static const String _kIllustrationCount = 'ai_illustration_count';
  static const String _kLegacyIllustrationCount = 46ai_storybook_page_count';

```

```

static const String _kLastLocalModelId = 'ai_last_local_model_id';

String _activeLocalModelId = ModelManager.defaultLocalModelId;
int _pointsBalance = 0;
int _illustrationCount = 0; // 0 = Auto
bool _anyLocalModelInstalled = false;

final Map<String, ModelInstallStatus> _installStatusByModelId = {};
final Map<String, double> _downloadProgressByModelId = {};
final Map<String, String> _currentDownloadFileByModelId = {};

AiModelProvider() {
  // ... 方法实现省略 ...
}

bool get loaded => _llmClient != null && _llmClient!.isAvailable;
String get activeLocalModelId => _activeLocalModelId;

bool get anyLocalTextInstalled => _anyLocalModelInstalled;
bool get localTextReady => loaded;

int get illustrationCount => _illustrationCount;

Future<void> setIllustrationCount(int value) async {
  // ... 方法实现省略 ...
}

Future<void> _checkAnyLocalModelInstallation() async {
  // ... 方法实现省略 ...
}

Future<String?> _findInstalledLocalModelId() async {
  // ... 方法实现省略 ...
}

Future<void> _ensureSelectedLocalModelInstalledIfAny() async {
  // ... 方法实现省略 ...
}

int get pointsBalance =>

Future<void> setPointsBalance(int value) async {
  // ... 方法实现省略 ...
}

final v = value == null ? null : (value < 0 ? 0 : value);
final prefs = await SharedPreferences.getInstance();

final override = _debugPointsOverride;

/// 从服务端同步积分余额（登录后用 userId，否则用 deviceId）

final deviceId = TencentApiClient.deviceId;
String.fromEnvironment('AIRREAD_API_PROXY_URL', defaultValue: '');
String.fromEnvironment('AIRREAD_API_KEY', defaultValue: '');

```

```

        final json = jsonDecode(resp.body);
        final balance = (json['balance'] as num?)?.toInt();

        final prefs = await SharedPreferences.getInstance();
        final localModelRaw = (prefs.getString(_kLastLocalModelId) ?? '').trim();

        final allowedPages = <int>{0, 4, 8, 12};
        int rawPages = prefs.getInt(_kIllustrationCount) ?? 0;

        ModelInstallStatus installStatusFor(String modelId) {
            // ... 函数实现省略 ...
        }

        double downloadProgressFor(String modelId) {
            // ... 函数实现省略 ...
        }

        String currentDownloadFileFor(String modelId) {
            // ... 函数实现省略 ...
        }

        bool isDownloadingModel(String modelId) {
            // ... 函数实现省略 ...
        }

        return false;

        final ok = await ModelManager.isModelInstalled(spec.id);

        final installed = await isLocalModelInstalled(modelId);
        throw StateError('本地模型未下载: ${ModelManager.displayNameFor(modelId)}');
        // ... 函数实现省略 ...
    }

    final c = _llmClient;

    void _markLocalInferenceUsed() {
        // ... 函数实现省略 ...
    }

    final prefs = await SharedPreferences.getInstance();

    final id = _downloadingModelId;

    final id = _downloadingModelId;

    // =====
    // 文件: ai/local_llm/llm_client.dart
    // 原始行数: 272 | 保留行数: 45
    // =====

    import 'dart:async';
    import 'dart:convert';
    import 'dart:io';

```

```

import 'package:flutter/foundation.dart';
import 'package:path/path.dart' as p;
import 'mnn_client.dart';
import 'model_manager.dart';

abstract class LlmClient {
  bool get isAvailable;
  String? get currentModel;

  Future<bool> initialize({String? model});
  Future<String> generate({
    required String prompt,
    int maxTokens = 512,
    double temperature = 0.7,
    double topP = 0.9,
    int topK = 40,
    double minP = 0.0,

// =====
// ... ai/local_llm/llm_client.dart 中间部分代码省略 (232 行) ...
// =====

  })

  @override
  Future<void> dispose() async {
    debugPrint('[LlmClientMnn] dispose');
    await _client.dispose();
  }
}

/// 创建适合当前平台的本地 LLM 客户端
LlmClient createLocalLlmClient() {
  if (Platform.isIOS || Platform.isAndroid) {
    // iOS 和 Android 使用 MNN
    return LlmClientMnn();
  } else {
    // 其他平台暂不支持
    throw UnsupportedError('Local LLM is only supported on iOS and Android.');
```

```

  }
}

// =====
// 文件: ai/local_llm/mnn_client.dart
// 原始行数: 256 | 保留行数: 61
// =====

import 'dart:async';
import 'dart:convert';
import 'dart:io';
import 'package:flutter/foundation.dart';
import 'package:flutter/services.dart';

class MnnClient {
```

```

static const MethodChannel _channel = MethodChannel('airread/local_llm');
static const EventChannel _eventChannel =
    EventChannel('airread/local_llm_stream');

bool _isInitialized = false;
String? _modelPath;

MnnClient();

bool get isAvailable => _isInitialized;
String? get modelPath => _modelPath;

void _debugLogModelInput({
    required String where,
    required String prompt,
}) {
    if (!kDebugMode) return;
    final trimmed = prompt.trimRight();
    const tailLen = 160;
    final tail = trimmed.length <= tailLen
        ? trimmed

// =====
// ... ai/local_llm/mnn_client.dart 中间部分代码省略 (200 行) ...
// =====

Future<String?> dumpConfig() async {
    if (!_isInitialized) {
        return null;
    }

    try {
        final result = await _channel.invokeMethod<String>('dumpConfig');
        return result;
    } catch (e) {
        debugPrint('[MnnClient] dumpConfig error: $e');
        return null;
    }
}

Future<void> dispose() async {
    try {
        await cancel();
    } catch (_) {}
    try {
        debugPrint('[MnnClient] dispose called modelPath=$_modelPath');
        await _channel.invokeMethod('dispose');
    } catch (_) {}
    _isInitialized = false;
    _modelPath = null;
}
}

```

```
// =====
// 文件: ai/hunyuan/hunyuan_text_client.dart
// 原始行数: 115 | 保留行数: 61
// =====

import '../tencentcloud/tencent_api_client.dart';
import '../tencentcloud/tencent_credentials.dart';
import '../tencentcloud/embedded_public_hunyuan_credentials.dart';

/// 流式输出返回的数据结构
class ChatStreamChunk {
  final String content;
  final String? reasoningContent;
  final bool isReasoning;
  final bool isComplete;

  ChatStreamChunk({
    required this.content,
    this.reasoningContent,
    this.isReasoning = false,
    this.isComplete = false,
  });
}

class HunyuanTextClient {
  static const String _host = 'hunyuan.tencentcloudapi.com';
  static const String _service = 'hunyuan';
  static const String _version = '2023-09-01';
  static const String _region = 'ap-guangzhou';

  final TencentApiClient _api;
  final TencentCredentials _credentials;

// =====
// ... ai/hunyuan/hunyuan_text_client.dart 中间部分代码省略 (59 行) ...
// =====

    region: _region,
    secretId: _credentials.secretId,
    secretKey: _credentials.secretKey,
    useProxy: !usingPersonalTencentKeys(),
    timeout: null,
    payload: {
      'Model': model,
      'Stream': true,
      'Messages': messages ??
        [
          {'Role': 'user', 'Content': userText},
        ],
      'EnableEnhancement': true,
      'ForceSearchEnhancement': true,
    },
  );
```

```

    await for (final chunk in stream) {
      yield ChatStreamChunk(
        content: chunk.content,
        reasoningContent: chunk.reasoningContent,
        isReasoning: chunk.isReasoning,
        isComplete: chunk.isComplete,
      );
    }
  }
}

// =====
// 文件: ai/hunyuan/hunyuan_image_client.dart
// 原始行数: 101 | 保留行数: 61
// =====

import 'dart:convert';
import '../tencentcloud/tencent_api_client.dart';
import '../tencentcloud/tencent_credentials.dart';
import '../tencentcloud/embedded_public_hunyuan_credentials.dart';

class HunyuanImageClient {
  static const String _host = 'aiart.tencentcloudapi.com';
  static const String _service = 'aiart';
  static const String _version = '2022-12-29';
  static const String _region = 'ap-guangzhou';

  final TencentApiClient _api;
  final TencentCredentials _credentials;

  HunyuanImageClient({
    TencentApiClient? api,
    required TencentCredentials credentials,
  }) : _api = api ?? TencentApiClient(),
       _credentials = credentials;

  String _truncateUtf8(String s, {required int maxBytes}) {
    if (maxBytes <= 0) return '';
    final raw = s.trim();
    final rawBytes = utf8.encode(raw);
    if (rawBytes.length <= maxBytes) return raw;

    final out = StringBuffer();
    int used = 0;

    // =====
    // ... ai/hunyuan/hunyuan_image_client.dart 中间部分代码省略 (45 行) ...
    // =====

    /// 查询任务状态
    /// 返回包含 JobStatusCode, ResultImage 等字段的 Map
    /// JobStatusCode:
    /// 1: 初始化
    /// 2: 处理中

```

```

/// 3: 生成中
/// 4: 失败
/// 5: 成功
Future<Map<String, dynamic>> queryTextToImageJob(String jobId) async {
  final resp = await _api.postJson(
    host: _host,
    service: _service,
    action: 'QueryTextToImageJob',
    version: _version,
    region: _region,
    secretId: _credentials.secretId,
    secretKey: _credentials.secretKey,
    useProxy: !usingPersonalTencentKeys(),
    timeout: const Duration(seconds: 60),
    maxRetries: 120,
    payload: {
      'JobId': jobId,
    },
  );
  return resp;
}
}

// =====
// 文件: presentation/widgets/ai_hud.dart
// 原始行数: 3875 | 保留行数: 90
// =====

import 'dart:async';
import 'dart:convert';
import 'dart:io';
import 'dart:math' as math;
import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:provider/provider.dart';
import 'package:shared_preferences/shared_preferences.dart';

import '../ai/local_llm/mnn_model_spec.dart';
import '../ai/local_llm/model_manager.dart';
import '../ai/reading/qa_service.dart';
export '../ai/reading/qa_service.dart' show QAStructChunk, QAType;
import '../ai/tencentcloud/embedded_public_hunyuan_credentials.dart';
import '../ai/tencentcloud/tencent_credentials.dart';
import '../ai/translation/translation_types.dart';
import '../core/theme/app_colors.dart';
import '../core/theme/app_tokens.dart';
import '../models/ai_chat_model_choice.dart';
import '../providers/ai_model_provider.dart';
import '../providers/translation_provider.dart';
import '../providers/qa_stream_provider.dart';
import 'glass_panel.dart';
import 'illustration_panel.dart';
import 'points_wallet.dart';

```

```

import 'ai_inference_top_row.dart';

export '../providers/ai_model_provider.dart' show ModelInstallStatus;

enum AiHudRoute {

  /// AI companion bottom sheet with in-panel navigation.
  ///
  /// Design goals:
  /// - Continuous features use unified Switch toggles.
  /// - Remove the top-right close icon; sheet dismissal is handled by outside tap / drag.
  /// - Each row has its own settings entry (chevron) and row-tap navigation.
  class AiHud extends StatefulWidget {
    final Color bgColor;
    final Color textColor;
    final AiHudRoute initialRoute;
    final String? initialQaText;
    final bool autoSendInitialQa;

    /// Feature enabled state (controls whether quick actions should appear).
    final bool translateEnabled;

    /// Active state (controls whether translation is applied to the reader content).
    final bool translateActive;

    final ValueChanged<bool>? onTranslateChanged;

    final bool readAloudEnabled;
    final ValueChanged<bool>? onReadAloudChanged;

    /// QA scope data (per-book)
    final String bookId;

    /// Reading context for AI QA (use effective/plain text, not raw HTML)
    final Map<int, String> chapterTextCache;
    final int currentChapterIndex;
    final int currentPageInChapter;
    final Map<int, List<TextRange>> chapterPageRanges;
    final String? illustrationChapterIdSuffix;
    final ValueChanged<int>? onChapterIllustrationsGenerated;

    final void Function(String, {bool isError})? onShowTopMessage;
    // ... 函数实现省略 ...
  }

  class _AiHudState extends State<AiHud> with TickerProviderStateMixin {
    final List<AiHudRoute> _stack = [AiHudRoute.main];
    bool _initialQaConsumed = false;

    AiHudRoute get _route => _stack.isEmpty ? AiHudRoute.main : _stack.last;

    void _push(AiHudRoute next) {
      // ... 函数实现省略 ...
    }
  }
}

```

```
void _pop() {  
  // ... 函数实现省略 ...  
}
```

```
void initState() {  
  // ... 函数实现省略 ...  
}
```

```
Widget build(BuildContext context) {
```